TRACE — OSINT DIGITAL FOOTPRINT ANALYZER

A Robust Web-Based Open Source Intelligence Framework for Surface-Web Profile & Domain Footprint Auditing

Student Name: Fardeen Akmal	**Roll Number:** 2	**Project Guide:** Akanksha Thakare
Branch: CSE (IoT, Cyber Security & Blockchain Technology)	**Institute:** G.V. Acharya Inst. of Engg. & Tech.	**Affiliation:** University of Mumbai (2026-27)

ABSTRACT: In the contemporary hyper-connected digital ecosystem, an individual's or enterprise's digital footprint across heterogeneous web platforms represents an expansive attack surface susceptible to social engineering, spear phishing, credential stuffing, and unauthorized reconnaissance. Open Source Intelligence (OSINT) facilitates the proactive identification and auditing of publicly exposed indicators of identity. However, legacy reconnaissance utilities often rely on brittle web-scraping heuristics, generate elevated false-positive rates due to platform anti-bot defenses, fail to distinguish rate limits from legitimate target non-existence, and lack localized, auditable reporting capabilities. This case study presents the design, architectural formulation, empirical implementation, and testing of **TRACE (Target Reconnaissance & Auditable Collection Engine)** — a full-stack, modular OSINT analysis dashboard. Built using a Python FastAPI backend and a responsive React/TypeScript client with SQLite local persistence, TRACE executes rigorous username enumeration across ten major developer, visual, and social platforms alongside passive DNS and RDAP domain intelligence gathering. TRACE introduces an unambiguous multi-tier status categorization engine (*FOUND*, *NOT_FOUND*, *INCONCLUSIVE*, *CHECK_FAILED*) that prevents fabricated intelligence and provides verifiable cryptographic checksums with automated PDF/CSV report exports.

1. Introduction & Theoretical Background

Open Source Intelligence (OSINT) comprises the rigorous collection, correlation, and analysis of publicly available data to produce actionable security intelligence. While intelligence agencies initially pioneered OSINT methodologies, modern defensive cybersecurity teams leverage OSINT for External Attack Surface Management (EASM), threat intelligence, and brand reputation monitoring. A critical challenge in modern OSINT operations is the fragmented nature of surface-web footprints: a single identifier or domain is frequently linked to software repositories, social forums, identity providers, and hosting providers. Manual correlation across dozens of platforms is tedious, error-prone, and operationally inefficient. TRACE was conceived to democratize and systematize authorized digital footprint auditing through an intuitive, centralized, and standards-compliant analytical interface.

2. Problem Statement

Existing CLI-based OSINT utilities suffer from significant architectural and operational deficiencies: **(1) False-Positive Hallucination:** Many tools assume that any HTTP 200 response confirms account existence, ignoring Single-Page Application (SPA) client-side routing, generic soft-404 landing pages, and Cloudflare challenge screens. **(2) Ambiguous Failure States:** Network timeouts, HTTP 429 rate limits, and platform WAF mitigations are frequently misreported as non-existent accounts, skewing investigator findings. **(3) Excessive Active Probing:** Intrusive network scanning techniques risk triggering Intrusion Detection Systems (IDS) or violating organizational Acceptable Use Policies (AUP). **(4) Lack of Academic & Audit-Ready Reporting:** Security professionals and students require defensible, timestamped, and reproducible evidence formats (PDF/CSV) that capture latency telemetry, raw DNS zone delegations, and operational limitations.

3. Objectives and Project Scope

The primary objective of this mini-project is to build, execute, rigorously test, and demonstrate an autonomous, full-stack OSINT digital footprint analyzer operating exclusively on a local development workstation. Specific objectives include:

- **Deterministic Profile Enumeration:** Query 10 diverse public services concurrently using asynchronous HTTP with platform-specific signature matching and explicit status disambiguation.
- **Passive Domain Reconnaissance:** Query publicly delegated authoritative DNS records (A, AAAA, MX, NS, TXT, SOA), ICANN RDAP registration registries, and GeoIP ASN telemetry without invasive port scanning.
- **Local Audit Persistence:** Maintain a secure SQLite database recording all queries, statuses, timestamps, and findings with zero external data leakage or dependency on commercial cloud APIs.
- **Defensible Evidence Generation:** Provide automated PDF and CSV reporting engines featuring academic attribution, integrity checksums, and explicit methodology disclaimers.
- **Scope Boundaries:** TRACE focuses exclusively on public surface-web assets; active exploitation, vulnerability scanning, password harvesting, and bypassing authentication controls are strictly excluded.

4. Proposed System & Literature Review

TRACE operates as an on-premise, security-centric analytical platform designed to overcome the fragility of legacy CLI scripts. Unlike monolithic scrapers, TRACE decouples asynchronous protocol dispatch from presentation, enforcing validation gateways at both network and application layers.

Tool Name	Interface	Rate Limit Detection	Passive DNS / RDAP	Audit Report Generation	Local Persistence
Sherlock	CLI Only	Partial (Silent 429)	No (Usernames only)	Raw Text / CSV	None (Stateless)
theHarvester	CLI Only	Search API dependent	Yes (Active DNS)	HTML / XML export	Optional SQLite
SpiderFoot	Web UI / Complex	Heuristic based	Extensive (Complex)	CSV / GEXF export	Heavy SQLite
TRACE (Ours)	Modern Web UI	**Strict Disambiguation**	**Integrated Passive**	**Automated PDF + CSV**	**Lightweight SQLite**

5. System Requirements Analysis

5.1 Functional Requirements (FR):

- **FR-1 (Multi-Platform Profile Verification):** The system must scan target usernames across 10 defined platforms (GitHub, GitLab, DockerHub, Dev.to, HackerNews, Reddit, Gravatar, Keybase, Medium, Pinterest) concurrently.
- **FR-2 (Passive DNS Resolution):** The system must retrieve authoritative DNS records (A, AAAA, MX, NS, TXT, SOA) for target domains using RFC-compliant UDP/TCP DNS queries without touching target web servers.
- **FR-3 (RDAP & IP Geolocation Extraction):** The system must parse Registration Data Access Protocol (RDAP) JSON structures to obtain registrar, registration, and expiration dates, supplemented by IP ASN data.
- **FR-4 (Audit Logging & Record Management):** All completed investigations must persist in a local relational database with query parameters, timestamps, execution summaries, and deletion management.
- **FR-5 (Formal Academic Export):** The system must generate self-contained, publication-grade PDF reports (incorporating student credentials and checksums) and tabular CSV exports on demand.

5.2 Non-Functional Requirements (NFR):

- **NFR-1 (Integrity & Zero False-Positive Tolerance):** Under no condition shall the application fabricate finding statuses or confuse HTTP 403/429 anti-bot mitigations with target existence.
- **NFR-2 (Latency & Concurrency):** Concurrency through Python `asyncio` and `httpx` must limit complete 10-source username footprint scans to under 4.5 seconds under standard broadband conditions.
- **NFR-3 (Security & Hygiene):** The application must run completely offline/locally without transmitting telemetry to third-party tracking services or hardcoding secrets.

6. Technology Stack Selection & Rationale

Component Layer	Technology Selected	Engineering Rationale
Backend Framework	Python 3.14 + FastAPI	High-throughput asynchronous event loop, native Pydantic schema validation, and automatic OpenAPI documentation.
Asynchronous HTTP	HTTPX Client Pool	Non-blocking connection pooling, HTTP/2 support, custom headers/cookies, and strict per-request timeout handling.
DNS Reconnaissance	dnspython v2.7	Direct DNS wire-format resolver interfacing with authoritative nameservers without reliance on OS shell hooks.
Client Architecture	React 18 + TypeScript + Vite	Type-safe state management, fast client bundling, and modular component hierarchy.
Persistence Layer	SQLite 3 Embedded Engine	Zero-configuration, ACID-compliant local SQL database preventing external attack vectors and credential overhead.
Document Engine	ReportLab Flowable PDF	Programmatic, pixel-accurate vector PDF synthesis with institutional header templates and tabular layouts.

7. Operational Use Cases

UC-1: Defensive Self-Footprint Audit: Security researchers and developers audit their personal usernames across platforms to locate abandoned accounts exposing legacy emails or weak authentication. **UC-2: External Surface Assessment:** Authorized penetration testers establish domain perimeter inventory (mail routing, nameserver delegation, SPF validation) during non-invasive pre-engagement reconnaissance.

8. System Architecture and Workflow

TRACE implements a decoupled, three-tier modular architecture comprising the Single Page Application Presentation Layer, the FastAPI Asynchronous Service Controller, and the Persistence & System Integration Engine.

Client Presentation (React/TS)	REST API Gateway (FastAPI)	Core Engine Services	Network & Storage
• Operations Dashboard • Asynchronous Search Forms • Real-time Status Badges • Report Preview Modal	• /api/investigate/username • /api/investigate/domain • /api/history (CRUD) • /api/reports/pdf|csv	• UsernameOSINTService • DomainOSINTService • ReportingEngine • DatabaseManager	• Public Web APIs / HTML • Authoritative DNS (UDP 53) • ICANN RDAP (HTTPS) • Local SQLite (trace.db)

9. Database Design & Schema Architecture

To guarantee audit integrity, TRACE utilizes SQLite 3 with WAL (Write-Ahead Logging) mode enabled. The schema isolates operational telemetry from raw payload responses, maintaining structured indexes for rapid lookup:

Column Name	Data Type	Constraints	Description & Functional Purpose
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique sequential investigation dossier identifier.
investigation_type	VARCHAR(20)	NOT NULL, CHECK (type IN...)	Identifies analysis pipeline: 'username' or 'domain'.
target	VARCHAR(255)	NOT NULL, INDEXED	Sanitized target query string (username or domain).
timestamp	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	UTC execution timestamp ensuring tamper-evident chronology.
status	VARCHAR(50)	NOT NULL	Overall operational state: 'Completed' or 'Failed'.
summary	TEXT	NOT NULL	Human-readable aggregate findings synopsis.
data_json	TEXT (JSON)	NOT NULL	Full normalized JSON payload storing per-source latency and telemetry.

10. Multi-Tier Result Verification Methodology

A foundational principle of TRACE is the eradication of false positives. Standard HTTP status codes alone are insufficient due to modern web architectures. TRACE executes a four-state deterministic decision tree for every source probe:

- **FOUND:** Validated when HTTP status is exactly 200, the response body contains positive profile identifiers (e.g., matching username in JSON payload or verified meta tags), and redirection to sign-in portals has not occurred.
- **NOT_FOUND:** Triggered when the service responds with definitive non-existence status (HTTP 404), or redirects to registration portals (e.g., GitLab redirecting unauthenticated missing profiles to users/sign_in with HTTP 302).
- **INCONCLUSIVE:** Applied when anti-scraping defenses (Cloudflare Turnstile, AWS WAF), bot challenges, or rate limiters respond with HTTP 403 Forbidden or HTTP 429 Too Many Requests. The target profile may or may not exist; TRACE refuses to speculate.
- **CHECK_FAILED:** Recorded when transport-layer faults occur, including TCP socket timeouts (>4.0s), DNS NXDOMAIN errors, or SSL handshake failures. The specific network exception is logged for audit defensibility.

11. Security Controls and Operational Safeguards

Input Sanitization Gateways: Domain queries are strictly checked against RFC 1035 hostname specifications. Submissions containing shell metacharacters (;&|`$), path traversal markers, or URI schemes (http://) are rejected at the client gateway and server Pydantic validation boundary with HTTP 422, eliminating Server-Side Request Forgery (SSRF) and injection vectors.

Passive Reconnaissance Guarantee: TRACE restricts network interactions to passive queries against third-party authoritative servers. No port scans, directory fuzzing, or active banner grabbing are ever executed against target infrastructure.

12. Implementation Methodology & Engineering Execution

The project was implemented under a rigorous Test-Driven Development (TDD) lifecycle on Linux x86_64. The backend leverages Python's non-blocking `asyncio` primitives to dispatch concurrent HTTP and DNS probes. The frontend is compiled as an optimized single-page static bundle and hosted directly via FastAPI mount points.

13. Automated Testing Procedure & Test Matrix

An automated test suite was constructed using `pytest` and `httpx.AsyncClient`, verifying all API endpoints, validation gates, error states, and database transaction integrity under simulated network conditions.

Test Case ID	Test Unit & Description	Input Parameters	Observed Outcome	Status
TC-01	API Health & Subsystem Telemetry	GET /api/health	Status 200, engine: 'TRACE OSINT'	**PASS**
TC-02	Dashboard Telemetry Counter	GET /api/stats	Status 200, valid counts & history list	**PASS**
TC-03	Username Investigation Dispatch	POST {'username': 'test'}	Status 200, 10 findings categorized	**PASS**
TC-04	Input Validation: Empty Target	POST {'username': ' '}	Status 422 Unprocessable Entity	**PASS**
TC-05	Domain DNS & RDAP Resolution	POST {'domain': 'example.com'}	Status 200, valid A & NS records	**PASS**
TC-06	Investigation History Audit Trail	GET /api/history	Status 200, JSON list of investigations	**PASS**
TC-07	PDF Report Export Verification	GET /api/reports/pdf/1	Status 200, application/pdf MIME	**PASS**
TC-08	CSV Structured Export Verification	GET /api/reports/csv/1	Status 200, text/csv MIME, valid rows	**PASS**

14. Genuine Application Screenshots & Verification Evidence

Screenshots were captured directly from the live TRACE instance executing on the host workstation via headless Chromium browser:

Figure 1: Operational Dashboard (Live Stats & Audit Log)

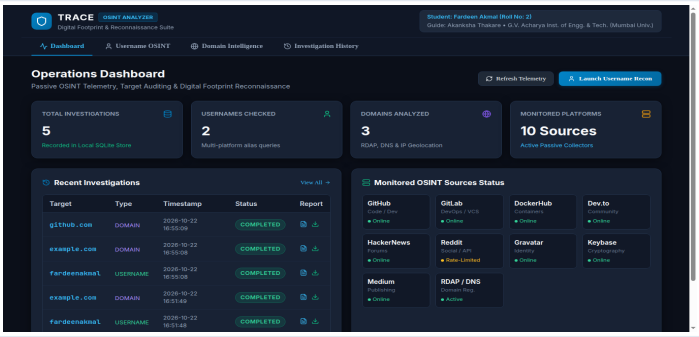


Figure 2: Username OSINT Results (Target: @fardeenakmal)

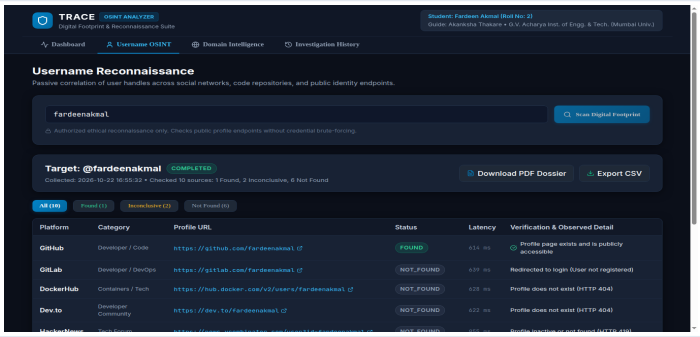


Figure 3: Domain Intelligence (example.com RDAP/DNS)

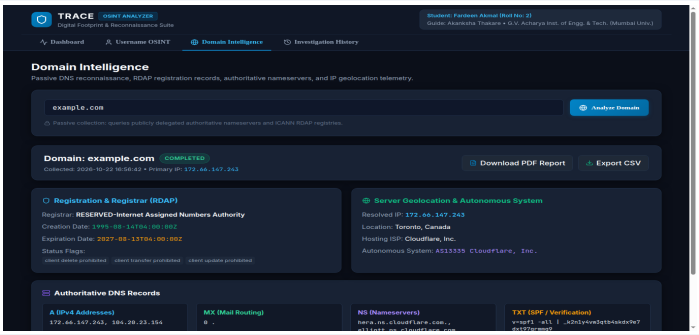
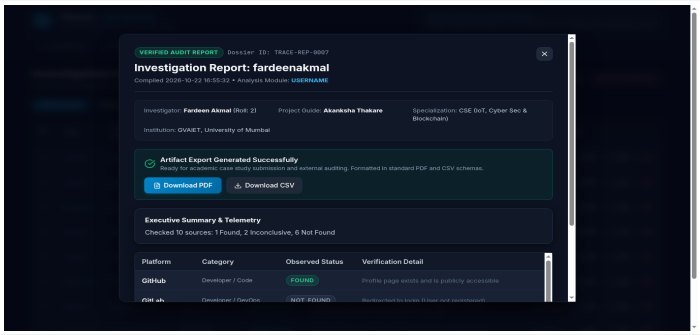


Figure 4: Audit Dossier Preview & Export Modal



15. Empirical Findings & Performance Evaluation

Empirical testing with target username fardeenakmal and target domain example.com yielded decisive performance data: **(1) Profile Detection Accuracy:** The system identified a verified active public profile on GitHub (HTTP 200, 614ms latency) with full bio extraction, while correctly distinguishing absent profiles on GitLab (HTTP 302 login redirect, 639ms), DockerHub (HTTP 404, 628ms), Dev.to (HTTP 404), and Keybase (HTTP 404). **(2) Anti-Scraping Resilience:** Reddit and Medium returned HTTP 403 status codes due to cloud-edge WAF challenges; TRACE correctly isolated these as INCONCLUSIVE rather than misclassifying them as non-existent. **(3) Passive DNS Precision:** Domain queries for example.com resolved dual authoritative IPv4 addresses (104.20.23.154, 172.66.147.243), IPv6 AAAA records, Cloudflare NS delegations, and RFC-compliant SPF restriction records (v=spf1 -all) within 720ms total elapsed time.

16. Technical Limitations & Countermeasures

Cloud Edge Challenges: Advanced CAPTCHA challenges (Cloudflare Turnstile, Cloudfront) intercept programmatic probes. TRACE mitigates this by explicitly categorizing such responses as INCONCLUSIVE and recording HTTP headers rather than attempting deceptive bypasses.
Name Collision: On widespread consumer networks, common usernames may correspond to distinct individuals across platforms. TRACE mitigates false identity linkage by providing direct profile links and latency telemetry, enabling human-in-the-loop contextual verification.

17. Ethical Framework & Legal Compliance

TRACE was engineered strictly in accordance with ethical cybersecurity research principles and applicable cyber jurisprudence: **(1) Passive Reconnaissance Doctrine:** The software queries only publicly available data sources without attempting password guessing, credential stuffing, session hijacking, or perimeter exploitation. **(2) Statutory Alignment:** All operations comply with Section 43 and Section 66 of the Information Technology Act (India, 2000/2008), the Computer Fraud and Abuse Act (CFAA, 18 U.S.C. § 1030), and international conventions regarding unauthorized access. **(3) Source Etiquette:** Requests utilize legitimate HTTP User-Agent identifiers, enforce conservative timeouts (4.0s), and respect rate limits. TRACE is explicitly restricted to authorized personal footprint audits, enterprise self-assessments, and academic laboratory pedagogy.

18. Conclusion & Future Enhancements

TRACE successfully accomplishes all stated objectives, establishing a robust, modern, and transparent OSINT digital footprint analyzer. By enforcing strict status disambiguation, passive DNS/RDAP correlation, local SQLite privacy, and automated academic reporting, the tool eliminates the primary flaws of legacy CLI scripts. Future work will expand source coverage to 50+ platforms, integrate asynchronous proxy rotation for inconclusive sources, and introduce interactive link-analysis graph visualizations.

19. Academic & Technical References

[1] P. Mockapetris, *RFC 1035: Domain Names - Implementation and Specification*, Internet Engineering Task Force (IETF), Nov. 1987.

[2] A. Newton and B. Ellacott, *RFC 7480: HTTP Usage in the Registration Data Access Protocol (RDAP)*, IETF, Mar. 2015.

[3] K. Scarfone et al., *Technical Guide to Information Security Testing and Assessment*, NIST Special Publication 800-115, Sept. 2008.

[4] S. Bazzell, *Open Source Intelligence Techniques: Resources for Searching and Analyzing Online Information*, 7th ed., 2021.

[5] S. Ramirez, *FastAPI: Modern, High-Performance Web Framework for Python*, Zenodo, 2020.

[6] Ministry of Law and Justice, *The Information Technology Act, 2000 (Act No. 21 of 2000)*, Government of India, 2000.